ScientoPy v 2.1.6, Installation and User Manual

Juan Pablo Ruiz Rosero
jpabloruiz@unicauca.edu.co

Contents

1 Installation **3**

2 Download the bibliometric dataset **3**

2.1 Download the dataset from Scopus . . . 3

2.2 Download the dataset from WoS . . . 4

3 Running the ScientoPy scripts **4**

3.1 Preprocessing . . . 5

3.2 Extract the top topics . . . 6

3.3 Analyze custom topics inside a criterion . . . 7

3.3.1 Asterisk (*) wildcard . . . 7

3.3.2 Evolution plot . . . 7

3.4 Additional options . . . 8

3.5 Topics growth indicators . . . 8

3.5.1 Average Growth Rate (AGR) . . . 8

3.5.2 Average Documents per Year (ADY) . . . 9

3.5.3 Percentage of Documents in Last Years (PDLY) . . . 9

3.5.4 h-index . . . 9

3.6 Finding trending topics . . . 9

3.7 Analysis based on the previous results . . . 10

3.8 Output files and directories . . . 10

4 ScientoPy graph types **12**

4.1 Time line graph . . . 12

4.2 Horizontal bars graph . . . 13

4.3 Horizontal bars trends 14
4.4 Evolution graph 14
4.5 Word cloud graph 15

5 BibTeX generation **17**

5.1 How it works 17
5.2 Usage 17
5.2.1 Using the cite keys 17
5.2.2 Running from the command line 17
5.2.3 Compiling the LaTeX document 18
5.2.4 Using the GUI 18

1 Installation

Requirements: Python 3.9 or higher.

1. For Windows download and install Python 3 (version 3.9 or higher) from: `https://www.python.org/downloads/`.
2. For Debian or Ubuntu run this command to install Python3 and tkinter:

   ```
   sudo apt-get install python3 python3-tk python3-pip python3-pil python3-pil.imagetk
   ```

3. Clone the repository:

   ```
   git clone https://github.com/jpruiz84/ScientoPy
   ```

4. Create a virtual environment and install dependencies:

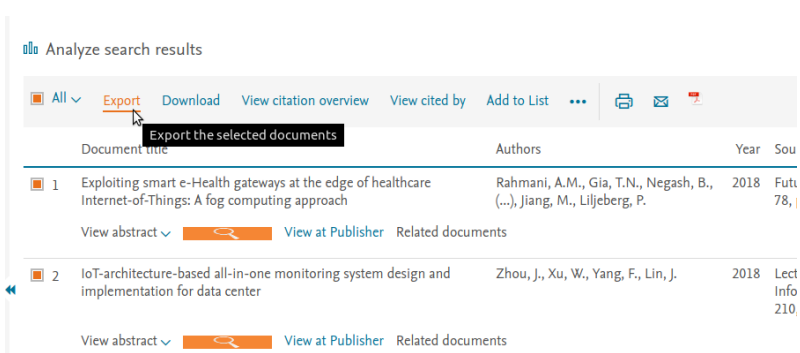
   ```
   python3 -m venv venv
   source venv/bin/activate        # Linux/macOS
   # venv\Scripts\activate          # Windows

   pip install --upgrade pip
   pip install -r requirements.txt
   ```

2 Download the bibliometric dataset

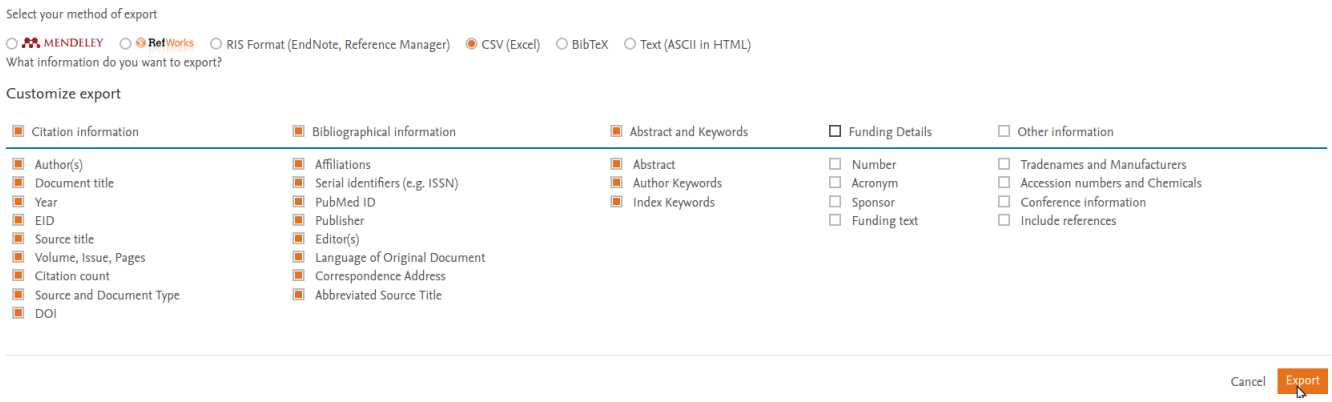
This section describes how to download the proper dataset from Scopus and WoS. Define a search criteria that will be used for Scopus and WoS. For this guide and for the example dataset we are using: "Bluetooth low energy"

2.1 Download the dataset from Scopus

1. Make your search with the defined search criteria for Article title, Abstract, Keywords.
2. Select all the results and click on Export:



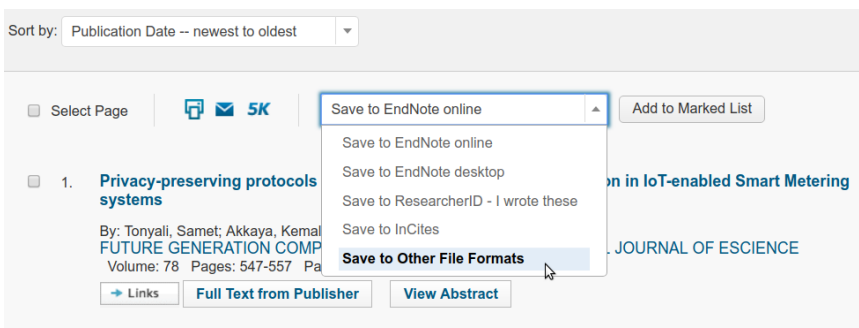
3. Select as method of export **CSV (Excel)**, and select the Customize export **Citation information, Bibliographical information, Abstract and Keywords**, then click on Export:



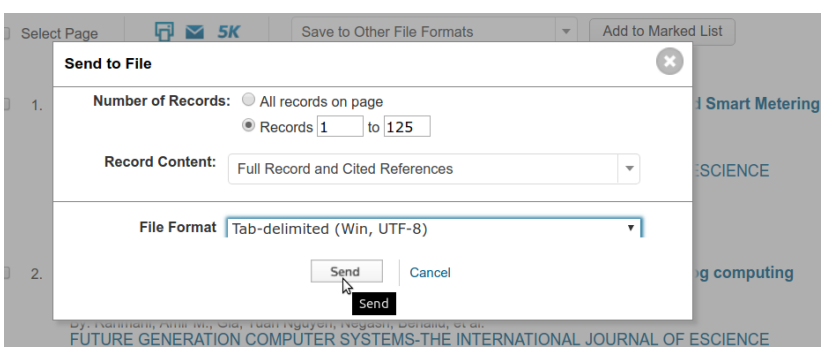
4. Save the file on the folder `/ScientoPy/dataIn`

2.2 Download the dataset from WoS

1. Make your search with the defined search criteria for Topic.
2. Select **Save in Other File Formats**



3. Select the number of records to download, on Record Contented select **Full Record and Cited References**, on File Format select **Tab-delimited (Win, UTF-8)**, and click on Send.



4. Save the file on the folder `/ScientoPy/dataIn`

3 Running the ScientoPy scripts

This section describes the ScientoPy scripts to preprocess and analyze the bibliometric dataset.

3.1 Preprocessing

First we need to preprocess the downloaded dataset. The preprocessing improves the dataset readability and precision through the following steps:

1. **Document type filtering:** By default, ScientoPy filters publications classified as: Conference Paper, Article, Review, Proceedings Paper, or Article in Press. Other document types (book chapters, short surveys, letters, notes, editorials, erratum, reports, etc.) are excluded.
2. **Field tags correlation:** WoS and Scopus use different field tags for their datasets. ScientoPy converts WoS two-character field tags to Scopus style fields and uses the Scopus format internally.
3. **Author's name normalization:** To ensure consistency between WoS and Scopus author name formats, ScientoPy applies the following normalizations:
   - Replace Scopus dot and comma separators with semicolons
   - Remove dots and commas from WoS and Scopus author fields
   - Remove accents marks and special characters
   - Convert the first author last name to upper case
4. **Duplicated removal:** ScientoPy detects duplicated documents based on the DOI, the normalized title, and the normalized first author last name. When duplicates are found between WoS and Scopus, ScientoPy keeps the WoS document and removes the Scopus one. For duplicated documents with different "cited by" counts, ScientoPy calculates the average times cited between the two entries and assigns it to the kept document.
5. **Times cited:** When a document is duplicated, the *times cited* field often does not match between databases. ScientoPy gets the average times cited between the two duplicated documents and sets it in the document that is going to be kept. This value is used later for h-index calculations.
6. **Country and institution extraction:** ScientoPy extracts the document's country from all author's affiliations (last section after the comma). Institution names are extracted from the first section before the comma in WoS documents' affiliation fields. Country names are normalized (e.g., "USA" → "United States", "UK" → "United Kingdom").

To preprocess the example dataset ("Bluetooth low energy" downloaded in April 2, 2026, located in dataInExample) run this command inside ScientoPy folder:

```
python3 preProcess.py dataInExample
```

Then, inside the folder `ScientoPy/dataPre` you will find the following files:

- **papersPreprocessed.csv:** this file contains the information of all papers after the pre-process. This file will be used by the others scripts as the input data.
- **PreprocessedBrief.csv:** this file briefs the pre-process statics results. The columns in this table are:
  - **Info:** description of the preprocessing statistic.
  - **Number:** count of documents.
  - **Percentage:** relative percentage. *Omitted papers by document type* percentage is relative to *Total loaded papers*. *Papers from WoS/Scopus* percentages are relative to *Total loaded papers*. *Total duplicated papers found* percentage is relative to *Total loaded papers*. *Removed duplicated papers from WoS* percentage is relative to *Papers from WoS*. *Removed duplicated papers from Scopus* percentage is relative to *Papers from Scopus*. *Duplicated documents with different cited by* percentage is relative to *Total duplicated papers found*. *Papers from WoS/Scopus* (after removal) percentages are relative to *Total papers after rem. dupl.*

- **Source, Conference Paper, Article, Review, Proceedings Paper, Article in Press, Total:** these columns show the document type breakdown per database source.

To find more options of the preprocessing script you can run:

```
python3 preProcess.py -h
```

3.2 Extract the top topics

With this script you can extract the top topics of a selected criterion. The ScientoPy criterion are described on Table 1:

Table 1: ScientoPy criterion description

Criteron	Description
author	Authors last name and first name initial
sourceTitle	Publication or journal name
subject	Research areas, only from WoS documents
authorKeywords	Author keywords
indexKeywords	Keywords generated by the index, from WoS {Keyword Plus}, and from Scopus {Indexed keywords}
bothKeywords	AuthorKeywords and indexKeywords are used for this search
abstract	Document abstract, for use with pre-defined topics and asterisk wildcard
documentType	Type of document
dataBase	Database where the document was extracted (WoS or Scopus)
country	Country extracted from authors affiliations
institution	Institution extracted from authors affiliations
institutionWithCountry	Institution with country extracted from authors affiliations

For example, to find the top author keywords you can run this script:

```
python3 scientoPy.py -c authorKeywords
```

This will generate a list with the top 10 topics on the selected criterion (in this case authorKeywords), with the number of documents per topic. Also, this script show the bar graph with the percentage of documents in the last years, and saves the quantitative results on the folder `ScientoPy/results`.

This script have more options like, save the plot on a file, or increase the number of topic results. For more information you can run:

```
python3 scientoPy.py -h
```

3.3 Analyze custom topics inside a criterion

If you want to make an analysis of custom topics, such as the two selected countries papers evolution, you can use the `scientoPy.py` script, with the option `-t`, to specify the topics:

```
python3 scientoPy.py -c country -t "United States; Brazil"
```

You can analyze any topic in any criterion. Put the topics on the `-t` argument. Divide the topics with the `;`. Also, you can integrate two or more topics in one, by dividing it with `,`. This is very useful for abbreviations and plural singulars, for example:

```
python3 scientoPy.py -c authorKeywords -t \
"WSN, Wireless sensor network, Wireless sensor networks; RFID, RADIO FREQUENCY IDENTIFICATION"
```

Note: The command is very long, for that reason the command was divided by `\`. If you have problems in Windows, remove the "änd put the command in one single line.

3.3.1 Asterisk (*) wildcard

You can use the asterisk wildcard to find phrases or words which starts or ends with the letters that you have inserted. For example, if you want to find "device", "devices", and "device integration", enter the following command:

```
python3 scientoPy.py -c authorKeywords -t "device*"
```

ScientoPy will print the topics found for the previous search:

```
Topics found for device*:
"Device Discovery;device-to-device synchronization;Device authentication;device-free;
device to device;Device driver;Device management;Device Independence;Device Identification"
```

You can use this information, to analyze each specific topic found, like this:

```
python3 scientoPy.py -c authorKeywords -t \
"Device Discovery;device-to-device synchronization;Device authentication;device-free; \
device to device;Device driver;Device management;Device Independence;Device Identification"
```

3.3.2 Evolution plot

Also, you can see the results with an evolution plot (add `-g evolution`). This option plots the accumulative documents, average documents per year (ADY), and percentage of documents in the last years, for example:

```
python3 scientoPy.py -c authorKeywords -t \
"WSN, Wireless sensor network, Wireless sensor networks; RFID, RADIO FREQUENCY IDENTIFICATION" \
-g evolution
```

This script have more options like, save the plot on a file, or others. For more information you can run:

```
python3 scientoPy.py -h
```

3.4 Additional options

The `scientoPy.py` script provides the following additional options:

- `--onlyFirst`: Only look in the first element of the topic, for example to analyze only the first author name, country or institution.
- `--graphTitle "Title"`: Put a title in the output graph.
- `--plotWidth N`: Set the plot width size in inches (default: 6.4).
- `--plotHeight N`: Set the plot height size in inches (default: 4.8).
- `-f "filter"` or `--filter "filter"`: Filter to be applied on a sub topic. For example, to extract institutions from the United States:

  ```
  python3 scientoPy.py -c institutionWithCountry -f "United States"
  ```

- `--pYear`: Present the results in percentage per year instead of documents per year.
- `--yLog`: Plot Y axes in log scale.
- `-s N` or `--skipFirst N`: To filter the first top N elements from the results.

3.5 Topics growth indicators

ScientoPy uses three different topic growth indicators to find trending topics and their relative/absolute growth. These indicators are included in the output results table.

3.5.1 Average Growth Rate (AGR)

The AGR is the average difference between the number of documents published in one year with the number of documents published in the previous year, inside a time frame. It indicates how the number of documents published for a topic has growth (positive number) or decline (negative number) on average. The AGR is calculated using the following equation:

$$AGR = \frac{\sum_{i=Y_s}^{Y_e} P_i - P_{i-1}}{(Y_e - Y_s) + 1} \tag{1}$$

where:

- AGR = Average growth rate
- Y_s = Start year, calculated as $Y_s = Y_e - (\text{WindowWidth} + 1)$
- Y_e = End year
- P_i = Number of publications on year i

The default WindowWidth is 2 years. Thus, if the end year is 2025, the AGR is the average growth rate between 2023 and 2025.

3.5.2 Average Documents per Year (ADY)

The ADY is an absolute indicator that represents the average number of documents published inside a time frame for a specific topic. The ADY is calculated using the following equation:

$$ADY = \frac{\sum_{i=Y_s}^{Y_e} P_i}{(Y_e - Y_s) + 1} \tag{2}$$

where:

- ADY = Average documents per year
- Y_s = Start year, calculated as described in Eq. 1
- Y_e = End year
- P_i = Number of publications on year i

3.5.3 Percentage of Documents in Last Years (PDLY)

The PDLY is a relative indicator that represents the percentage of the ADY relative to the total number of documents for a specific topic. The PDLY is calculated using the following equation:

$$PDLY = \frac{\sum_{i=Y_s}^{Y_e} P_i}{(Y_e - Y_s + 1) \times TND} \times 100\% \tag{3}$$

where:

- $PDLY$ = Percentage of documents in last years
- Y_s = Start year, calculated as described in Eq. 1
- Y_e = End year
- P_i = Number of publications on year i
- TND = Total number of documents

3.5.4 h-index

The h-index is extracted for each analyzed topic. A topic has an h-index of h if h of its documents have been cited at least h times. The h-index is calculated using the *times cited* field from the preprocessed dataset.

3.6 Finding trending topics

ScientoPy can find trending topics by looking at the top author keywords based on the highest AGR (described in the "Topics growth indicators" section above).

To find the top trending topics on author keywords criterion, you can run the following script:

```
python3 scientoPy.py -c authorKeywords --trend --startYear 2015 --endYear 2025 \
--windowWidth 2  --agrForGraph -g evolution
```

This script will find the top 200 topics, then it calculates the AGR for the last 2 years (`--windowWidth 2`). Finally, the 200 top topics are sorted from the highest AGR in the last 2 year period to the lower. The first 3 AGR topics are filtered (they correspond to the keyword Bluetooth low energy), and the next 10 topics are graphed in an evolution plot.

3.7 Analysis based on the previous results

ScientoPy generates an output file with all the output documents from the last run script. For example if we run the command:

```
python3 scientoPy.py -c country -t "Canada" --noPlot
```

ScientoPy will create a documents output file (`results/papersPreprocessed.csv`) with all documents that have authors with affiliation in Canada. This output file can be used by ScientoPy to perform an analysis based on this, in that way if we run the following command with the option `-r` or `--previousResults` after the previous one to analyze based on the previous results:

```
python3 scientoPy.py -c authorKeywords -r -g bar
```

we will obtain the top author keywords from papers where the author affiliation correspond to Canada. Also, we can run the following command to know which are the countries that have more common documents with Canada:

```
python3 scientoPy.py -c country -r -g bar
```

Note: the ScientoPy documents output file is only generated when the `-r` or `--previousResults` is not used. In that way, if we run many times a ScientoPy command with this option, the documents output file will not overwritten.

3.8 Output files and directories

After run some ScientoPy commands or after run all the example commands by executing the script `exampleGenerateGraphs.sh` you will find the following folder and files structure:

```
ScientoPy
├── dataInExample
├── dataPre
│   ├── papersPreprocessed.csv
│   └── PreprocessedBrief.csv
├── graphs
├── Manual
└── results
    ├── AuthorKeywords.csv
    ├── AuthorKeywords_extended.csv
    └── papersPreprocessed.csv
```

These folders and output files are described bellow:

- **dataInExample:** contains Scopus and WoS example data set for the search criteria "Bluetooth low energy" downloaded in April 2, 2026. This is the input example for preprocess script.
- **dataPre:** output folder for the preprocess results, and input folder for scientoPy script.

- **papersPreprocessed.csv:** preprocesed papers data with all input documents merged, filtered, and duplication removed. This is the input file that scientoPy script uses.
- **PreprocessedBrief.csv:** preproceses brief table that shows the preprocess results related to total papers found per data base, the omitted papers, the duplicated papers count per data base, and the total number of papers per paper type (Conference paper, article, review...)
- **graphs:** graphs output folder for preprocess and scientoPy scripts
- **Manual:** folder with the pdf manual and example paper with scientoPy commands highlighted used for graph and tables generation.
- **results:** output folder for scientoPy result output files
- **AuthorKeywords.csv:** scientoPy output file for the selected criterion (in this case authorKeywords) that shows the top topics or the custom topics with the following columns:
  - **Pos.:** Position in the ranking.
  - **[Criterion]:** The topic name for the selected criterion (e.g., AuthorKeywords, Country, Author, etc.).
  - **Total:** Total number of documents for this topic.
  - **AGR:** Average Growth Rate, the average difference in documents between consecutive years inside the window period. See Eq. 1.
  - **ADY:** Average Documents per Year, the average number of documents published per year inside the window period. See Eq. 2.
  - **PDLY:** Percentage of Documents in Last Years, the percentage of ADY relative to the total number of documents. See Eq. 3.
  - **hIndex:** h-index for the topic. A topic has h-index h if h of its documents have been cited at least h times.
  - **[Year columns]:** Number of documents published in each year within the analysis range.
- **AuthorKeywords_extended.csv:** scientoPy output file for the selected criterion (in this case authorKeywords) that show the top or custom topics with the documents related to each one.
- **papersPreprocessed.csv:** inside the results folder, this file contains the output papers from the last scientoPy used script. This is used as an input for scientoPy script when it use the option `-r` or `--previousResults`

4 ScientoPy graph types

ScientoPy has 5 different ways to graph the results described on Table 2.

Table 2: ScientoPy output graphs types

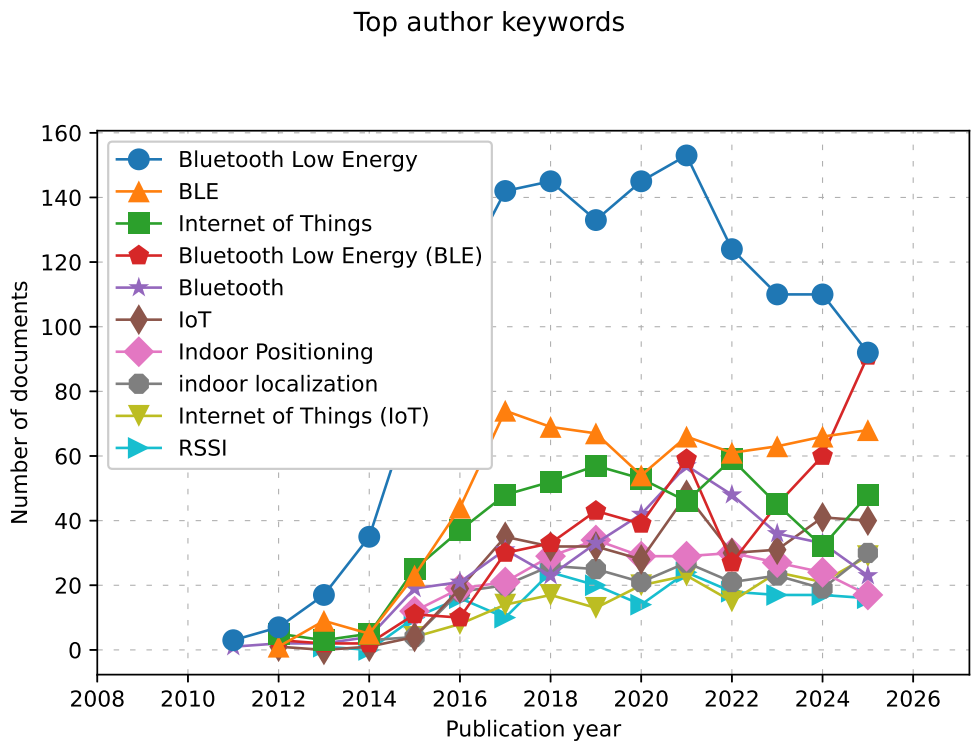
Graph type	Argument	Description
Time line	`-g time_line`	Graphs the number of documents of each topic vs the publication year
Horizontal bars	`-g bar`	Graphs the total number of documents of each topic in horizontal bars
Horizontal bars trends	`-g bar_trends`	Graphs the total number of documents of each topic in horizontal bars, with the percentage of document published in the last years
Evolution	`-g evolution`	Graphs two plots: on the left, the accumulative number of documents (logarithmic scale) vs. the publication year, where the beginning of each line shows when the topic research started and the final value represents the total number of documents; on the right, a scatter plot of the Average Growth Rate (AGR) on the Y-axis vs. the Percentage of Documents in Last Years (PDLY) on the X-axis, allowing comparison of absolute and relative growth for each topic
Word cloud	`-g word_cloud`	Generate a word cloud based on the topic total number of publications

Below are showed some examples of these graphs types, with the command used.

4.1 Time line graph

Command:

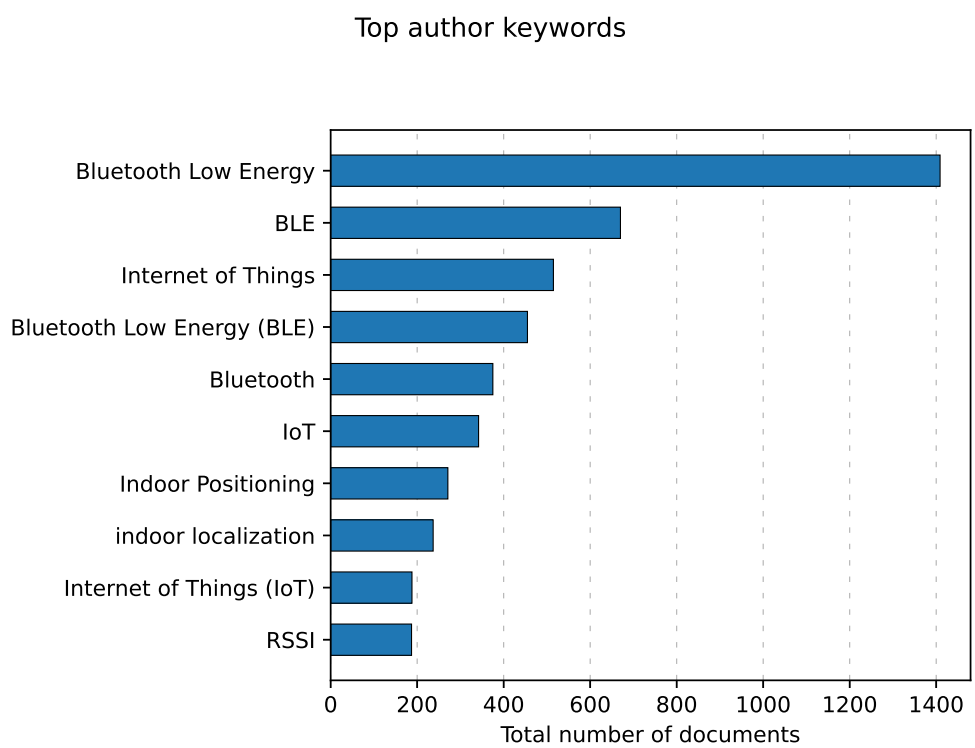
```
python3 scientoPy.py -c authorKeywords --startYear 2015 --endYear 2025 -g time_line
```



4.2 Horizontal bars graph

Command:

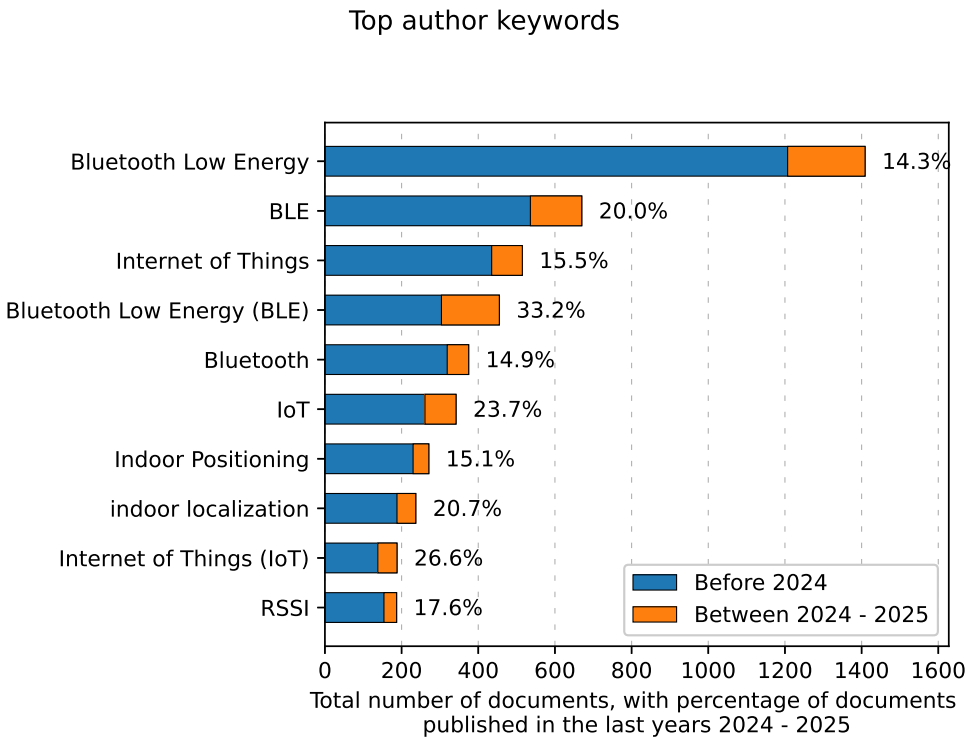
```
python3 scientoPy.py -c authorKeywords --startYear 2015 --endYear 2025 -g bar
```



4.3 Horizontal bars trends

Command:

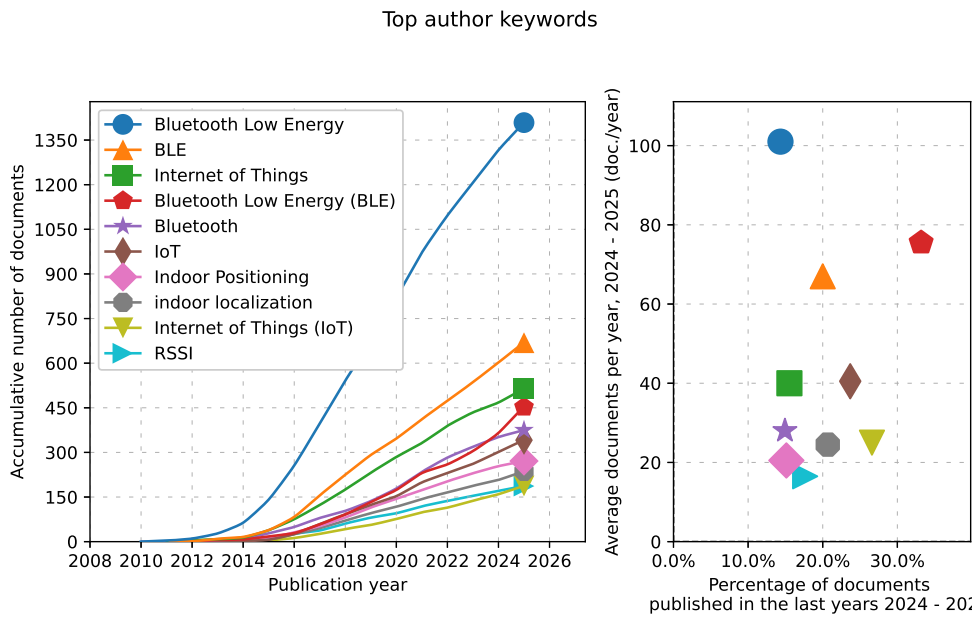
```
python3 scientoPy.py -c authorKeywords --startYear 2015 --endYear 2025 -g bar_trends
```



4.4 Evolution graph

Command:

```
python3 scientoPy.py -c authorKeywords --startYear 2015 --endYear 2025 -g evolution
```



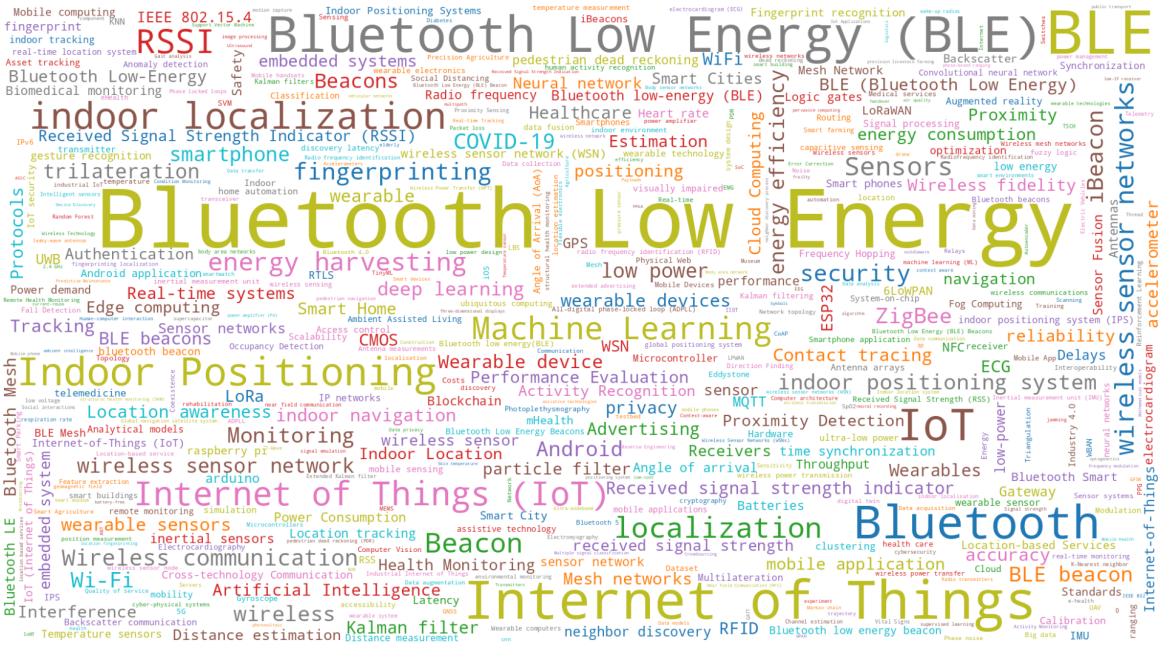
The evolution graph shows two panels:

- **Left panel (Accumulative documents):** The X-axis represents the publication year and the Y-axis the accumulative number of documents (in logarithmic scale). The beginning of each line indicates when the topic research started, and the final value of each line represents the total number of documents published for that topic.
- **Right panel (AGR vs. PDLY):** The Y-axis represents the AGR (Average Growth Rate) for the specified window period, and the X-axis represents the PDLY (Percentage of Documents in Last Years). Topics with higher AGR have the highest absolute growth, and topics with higher PDLY have the highest relative growth. This allows identifying which topics are trending (high AGR and high PDLY) versus which are established but declining.

4.5 Word cloud graph

Command:

```
python3 scientoPy.py -c authorKeywords --startYear 2015 --endYear 2025 -l 500 -g word_cloud
```


Bluetooth Low Energy (BLE)
BLE
Bluetooth Low Energy
Internet of Things (IoT)
Internet of Things
Indoor Positioning
Machine Learning
indoor localization
Bluetooth
IoT
localization
Beacon
RSSI
IEEE 802.15.4
Received Signal Strength Indicator (RSSI)
smartphone
trilateration
fingerprinting
energy harvesting
Real-time systems
deep learning
Edge computing
Tracking
Sensor networks
BLE beacons
Wearable device
Performance Evaluation
Activity Recognition
Location awareness
indoor navigation
Monitoring
wireless sensor network
Indoor Location
Android
particle filter
Received signal strength indicator
Wearable sensors
Wireless communication
Wi-Fi
Artificial Intelligence
wireless
Kalman filter
Distance estimation
Temperature sensors
Interference
Health Monitoring
Mesh networks
neighbor discovery
RFID
Bluetooth low energy beacon
received signal strength
mobile application
accuracy
BLE beacon
Location-based Services
Wearables
Receivers time synchronization
Proximity Detection
Advertising
privacy
MQTT
Contact tracing
ECG
indoor positioning system
reliability
Delays
ZigBee
ESP32
GLOWPAN
navigation
security
Sensors
energy consumption
Proximity
Wireless fidelity
wireless sensor network
iBeacon
Sensor Fusion
Antennas
accelerometer
Internet-of-Things
electrocardiogram
Cloud Computing
energy efficiency
Logic gates
BLE (Bluetooth Low Energy)
Backscatter
Synchronization
Mesh Network
Convolutional neural network
Augmented reality
LoRaWAN
Routing
Signal processing
optimization
low energy
Smart phones
Bluetooth beacons
Frequency Hopping
Fog Computing
indoor positioning system (IPS)
NFC
Bluetooth Smart
Gateway
wearable sensor
Standards
Calibration
IMU
Big data
Phase noise
Wireless mesh networks
fuzzy logic
Wireless sensors
capacitive sensing
Smartphone application
Antenna arrays
Received Signal Strength (RSSI)
Computer architecture
Industry 4.0
Triangulation
Energy
Neural networks
Throughput
Batteries
Clustering
health care
cybersecurity
Cloud
k-Nearest neighbor
Activity Monitoring
Supervised learning
Channel estimation
Visual Sign
Wireless power transfer
Radio transmitters
Error Correction
Noise
Fraity
Smart farming
Data communication
Bluetooth Low Energy (BLE) Beacons
Microcontroller
Direction Finding
Eddystone
Interoperability
Mobile App
Training
Scanning
Calibration
Wireless communications
System-on-Chip
Relays
machine learning (ML)
Mobile Devices
Physical Web
Museum
Radio frequency identification (RFID)
Mesh
Smart devices
Wireless Power Transfer (WPT)
Data transfer
Data collection
Smartphones
Heart rate
power amplifier
Estimation
Healthcare
Neural network
Smart Cities
Radio frequency
Bluetooth low-energy (BLE)
Indoor Positioning Systems
iBeacons
Beacons
embedded systems
pedestrian dead reckoning
WiFi
Wireless sensor network (WSN)
COVID-19
positioning
wearable
visually impaired
GPS
Angle of Arrival (AoA)
Location estimation
Physical Web
low power
performance
wearable devices
Smart Home
Ambient Assisted Living
Occupancy Detection
Access control
Scalability
CMOS
WSN
Microcontroller
Bluetooth low energy(BLE)
Communication
Costs
Blockchain
IP networks
LoRa
telemedicine
Social interaction
Photoplethysmography
mHealth
Bluetooth Low Energy Beacons
Wireless Sensor Network (WSN)
Hardware
ultra-low power
Throughput
Wireless power transmission
Batteries
Smart City
assistive technology
Electromyography
Computer Vision
Dataset
Multilateration
Latency
Servers
Quality of service
mobility
cyber-physical systems
5G
Backscatter communication
Wearable computers
Distance measurement
Phase noise
Big data
Signal strength

5 BibTeX generation

ScientoPy can automatically generate a BibTeX bibliography file from a LaTeX document. This feature matches `\cite{}` keys used in the LaTeX file against the preprocessed dataset, and produces a `.bib` file with the corresponding entries.

5.1 How it works

The `generateBibtex.py` script performs the following steps:

1. Reads the input LaTeX file and extracts all `\cite{key}` references from the document body (between `\begin{document}` and `\begin{thebibliography}`).
2. Loads the preprocessed dataset (`dataPre/papersPreprocessed.csv`).
3. Matches each citation key against the EID field of the papers in the dataset.
4. Generates BibTeX entries for matched papers:
   - `@Article{}` for journals, reviews, and articles in press.
   - `@Inproceedings{}` for conference papers and proceedings papers.
5. Handles author name formatting, converting Scopus and WoS formats into BibTeX's `and`-separated format.
6. Escapes LaTeX special characters (`&`, `_`) in titles and other fields.
7. Saves the output file next to the input file, with the suffix `_bibliography.bib`. For example, `myPaper.tex` produces `myPaper_bibliography.bib`.

5.2 Usage

Prerequisites: The preprocessing step must be completed first, so that the file `dataPre/papersPreprocessed.csv` exists.

5.2.1 Using the cite keys

In your LaTeX document, use the paper's EID as the cite key. The EID can be found in the extended results file (`results/*_extended.csv`) under the **EID** column. For example:

```
\cite{WOS:000363238300013}
\cite{WOS:000371717205060, WOS:000380412500076}
```

5.2.2 Running from the command line

To generate the BibTeX file, run:

```
python3 generateBibtex.py latexExample/article_example.tex
```

This will create the file `latexExample/article_example_bibliography.bib`.

5.2.3 Compiling the LaTeX document

In your LaTeX document, reference the generated bibliography:

```
\bibliographystyle{unsrt}
\bibliography{article_example_bibliography}
```

Then compile with the standard LaTeX/BibTeX cycle:

```
pdflatex article_example
bibtex article_example
pdflatex article_example
pdflatex article_example
```

A Makefile is provided in the `latexExample/` folder that automates this process. Run `make` inside that folder to generate the bibliography and compile the PDF in one step.

5.2.4 Using the GUI

In the ScientoPy graphical interface, click the **Generate BibTeX** button on the Analysis tab. This will open a file dialog to select your LaTeX file, generate the bibliography, and open the resulting `.bib` file.